

AGRITRACE INNOVATION WHITEPAPER

Cryptographic Off-Chain Hashing & On-Chain Supply Chain Ledger Storage Engine

Executive Summary

In modern supply chains, consumer trust depends entirely on data integrity. However, storing large quantities of transaction metadata directly on public blockchains (like Ethereum or Polygon) is financially prohibitive due to transaction network gas fees. Conversely, keeping all records in centralized databases leaves the system vulnerable to unauthorized database alterations. AgriTrace introduces a unique hybrid hashing architecture: the crop batch data hash is computed using off-chain data and written immutably to the Polygon blockchain smart contract. The raw metadata and image file assets remain off-chain. This creates an un-hackable validation loop at a fraction of the cost.

1. The Core Innovation: Dual-Ledger Architecture

The system operates on a dual-ledger layout that combines off-chain speed and capacity with on-chain security:

- **Off-Chain Capacity (PostgreSQL):** Raw crop registration parameters (Crop Name, Harvest Date, Farming Method, Location, Quantity) along with high-resolution image upload binaries are stored in a fast PostgreSQL relational database. This allows rapid querying, filtering, and report generation.
- **On-Chain Immutability (Polygon Layer 2):** To protect this data, AgriTrace does not write individual fields to the blockchain. Instead, it aggregates the core parameters, computes a single 32-byte cryptographic SHA-256 fingerprint (hash), and transmits this fingerprint to the deployed smart contract. The hash represents an immutable seal on the crop record.

2. Cryptographic Hashing Logic

The cryptographic crop fingerprint (Data Hash) is computed deterministically. The input string is constructed by concatenating the core parameters separated by pipe delimiter characters. Additionally, rather than including the entire image binary, the system computes the SHA-256 hash of the uploaded crop image file itself (Image File Hash). This ensures that if the image file is altered or replaced on the server, the validation check fails.

Hash Inputs Formula:

```
image_file_hash = SHA256(image_binary_content) input_string =  
f"{crop_name}|{harvest_date}|{farming_method}|{image_file_hash}" data_hash =  
SHA256(input_string.encode('utf-8'))
```

3. Real-Time Tamper Detection Engine

When a buyer scans the QR code or queries the crop status, the backend performs a real-time audit:

1. **Metadata Retrieval:** Fetches the crop batch record from the local PostgreSQL database.

2. ****On-Chain Retrieval:**** Queries the Polygon Smart Contract using the unique Batch ID to retrieve the original registered hash.
3. ****Re-Hashing:**** Re-calculates the SHA-256 hash based on the current values in the database and the image file currently stored on disk.
4. ****Comparison Verification:**** Compares the two hashes. If they match, the status is returned as ****AUTHENTIC****. If a database administrator or hacker altered the farming method from 'Organic' to 'Chemical', or replaced the crop image with a different one, the recomputed checksum will mismatch, returning ****TAMPERED**** status.

Step	Database Values	On-Chain Hash Signature	Audit Result
1. Crop Registered	Wheat, 2026-07, Organic, Image A	Writes Hash A	AUTHENTIC
2. Unauthorized Change	Wheat, 2026-07, Chemical , Image A	Immutable Hash A	TAMPERED (Calculated Hash B != Hash A)

4. Solidity Smart Contract Structure

The Polygon smart contract is optimized for minimal gas consumption, utilizing a mapping structure indexed by the unique string-based Batch ID. This enables O(1) hash writes and lookups.

```
contract CropTrace { struct BatchRecord { string dataHash; uint256 timestamp; bool
registered; } mapping(string => BatchRecord) private batches; function registerBatch(string
memory _batchId, string memory _dataHash) public { require(!batches[_batchId].registered,
"Batch already registered"); batches[_batchId] = BatchRecord(_dataHash, block.timestamp,
true); } }
```

5. Economic & Technical Value Matrix

Metric	Traditional Blockchain Storage	AgriTrace Hybrid Architecture
Gas Fees / Writes	Extremely high (linear to data size)	Minimal (fixed 32-byte hash cost)
Write Latency	Slow (requires network block mining)	Instant DB writes (writes to chain in background)
Image Binary Security	Stored on expensive decentralized storage (IPFS)	Stored locally; audited via immutable file hash signature